🎮 Learn Game Development

Fulllion Creative Works

Links, video titles, prices and license terms were checked on November 7, 2026, and each video's description here was checked against what the video actually says. They change, so confirm them on each official page.

Contents

1. How to Start
2. Choosing an Engine
3. Art and Assets
4. How Games Handle Performance
5. Fan Remakes, Mods and IP Etiquette
6. Game Jams
7. Your First Game: a Scope Checklist
8. Shipping Your Game
9. Publishing on itch.io and Steam
10. Starting a Game Company: the Basics
11. Keep Learning: Courses, Docs, Talks and Postmortems
12. Common Mistakes
13. Sources

---

1. How to Start

You don't need to pick the "perfect" engine or language first. The skills that matter most carry over between them: the game loop, input, movement, collisions, states and menus, and finishing what you start. Pick one of the courses below, follow it to the end, then make something of your own.

1.1 CS50's Introduction to 2D Game Development (Harvard, free)

The old edX link for "CS50's Introduction to Game Development" no longer works. The course now lives at **cs50.harvard.edu/games**, under the name **CS50's Introduction to 2D Game Development**.

- **What it covers:** eight weeks, each built around a classic game: Pong, Flappy Bird, Breakout, Match 3, Super Mario Bros., Legend of Zelda, Angry Birds and Pokémon, then a final project of your own. It teaches 2D graphics, animation, sound and collision detection.
- **Language:** Lua, with the free LÖVE 2D framework.
- **Prerequisites:** CS50x, or some experience in any programming language. If you've never coded, start with CS50x (see Computer Science Fundamentals).

- **Cost:** free through CS50's OpenCourseWare. The course FAQs describe a **free CS50 certificate** (you need at least 70% on each required assignment) and an optional **paid edX verified certificate**. When checked, the certificate page itself said "coming soon", so confirm the details on the FAQs before you start.
- **Deadline:** the FAQs give the current course deadline as the end of June 30, 2027 (UTC), and work at your own pace until then.
- **The older version:** the original course, which added 3D projects in Unity, was retired on June 30, 2024. Its materials remain online at cs50.harvard.edu/games/2018 for reference only.

1.2 "Pokémon Coding Tutorial" (freeCodeCamp.org)

"Pokémon Coding Tutorial - CS50's Intro to Game Development" (freeCodeCamp.org, about 2 hours) is Lecture 7 of the original 2018 CS50 game course, taught by Colton Ogden and posted in full. It doesn't build a whole Pokémon game. It walks through the code of a small Pokémon-style "simulator" in Lua and LÖVE: you walk a grid, meet wild creatures in tall grass, fight turn-based battles and level up, forever. The ideas it teaches carry over to any language:

- **A state stack.** Instead of one active state, states sit on a stack. Every state is drawn, bottom to top, but only the top one gets input and updates. So a dialogue box can sit on top of the overworld, which stays visible underneath.
- **Transitions as states.** A fade to white or black is its own small state, pushed on top and given a color, rather than code buried inside every screen.
- **Callbacks to chain events.** A dialogue box or fade takes a function to run when it finishes. That's how a battle plays out as a sequence: a message, then a flash, then the health bar draining, then the next message.
- **Grid movement.** The player's tile position changes at once, the sprite then slides (tweens) to it, and input is ignored until it arrives.
- **GUI building blocks.** Panels, nine-patch borders, text boxes that page through long text, selection menus whose items run callbacks, and progress bars reused for both health and experience.
- **Stats in data.** Each creature has base stats plus random "individual values", rolled against on every level-up, and the creature list is plain data, so adding one takes a few lines.
- **A small shader** that flashes a sprite white when it's hit.

Jump to:

- Pokémon and the RPG genre, then a demo of the finished project
- The state stack, and callbacks to chain events
- Grid-based movement and random encounters
- GUI: panels, text boxes, menus and progress bars
- Stats, individual values and leveling up
- The battle, step by step, and the assignment: a menu showing which stats went up

The last few minutes preview the next lecture, a 3D game in Unity, which belongs to the retired 2018 version of the course (Section 1.1).

1.3 freeCodeCamp's full courses

Both are free on the freeCodeCamp.org YouTube channel, with a written introduction on freeCodeCamp's site.

Course	Instructor	Length	What you build
Create an Arcade-Style Shooting Game with Python and PyGame (November 2022)	Pete LeMaster	About 2 hours	A shooting-gallery game: a gun that follows the mouse, waves of enemies, scoring, levels, pause and main menus, high scores saved to a file, and sound
Create a 3D RPG Game with Godot (June 2023)	Omar Zaki	About 5 hours	A 3D role-playing game: free assets, a camera, GridMap levels, animation trees, particle torches, enemy AI, an inventory, a shop, and saving and loading

Two version notes:

- The Godot article doesn't say which Godot version it uses. Godot has changed a lot between versions, so if a menu or function name doesn't match, check the version shown in the video and read the Godot documentation for your version. freeCodeCamp keeps adding newer Godot 4 courses to its YouTube channel.
- The Pygame course uses the original **pygame** library. Its community fork, **pygame-ce** ("Community Edition"), is made by former core developers and releases more often. Both use `import pygame`, so most tutorials work with either. Install one or the other in a project's virtual environment, not both:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install pygame-ce
```

On macOS or Linux, activate with `source .venv/bin/activate`. Python and Node Tool Environments explains virtual environments, and Safer Package Installs with npm and pip covers checking a package before you install it.

Another Pygame course: "The ultimate introduction to Pygame" (Clear Code, about 3 hours 48 minutes) builds a small endless-runner game, using it to teach every major part of Pygame: the game loop, frame rate, rectangles and collisions, player input, jumping and gravity, score, timers, animation, sprite classes and sound. It assumes you already know basic Python functions and simple classes. Coding Practice and Interview Prep lists "jump to" links for each of those topics. The video installs the original `pygame`; if you switch to pygame-ce, uninstall `pygame` first.

1.4 GameMaker, and "Start Making Games"

GameMaker is an engine built for 2D games. It's free to download and use for non-commercial projects, and you pay only when you want to sell a game (Section 2). You can write code in its own language, GML, or build logic visually with GML Visual. Its official tutorials and manual are free.

"Start Making Games" (Pirate Software, under a minute, March 2022) is a clip from a stream, in which a viewer says an empty project feels intimidating next to a finished game's. The answer is a concrete first step, in GameMaker:

1. Create a **blank project**. It's supposed to be empty.
2. Search for one small thing, such as "how do I make a character move in GameMaker".
3. Create your **first object**: right-click the **Objects** group in the asset browser and create one. An object is any "thing" in the game, a player, a lamp or a door, and you can have as many as you need. The clip mentions that the studio's own game, *Heartbound*, has a few thousand.

The point is to stop comparing your blank project with a finished one, and solve one small problem at a time. The description puts it as "Seriously, do it."

1.5 "Programming an entire game from scratch" (randy)

"Programming an entire game from scratch" (randy, 6 min, May 2023) follows an indie developer building a game without a ready-made engine. It's a casual devlog with strong language, and the title is partly a joke: he admits early on that "from scratch" means starting from the modules that come with his language. What he actually builds:

- **His own renderer,** on Direct3D 11: first the classic "hello triangle", then two triangles as a square (a **quad**) with a texture sampled onto it, which becomes a sprite.
- **Simple physics:** dropping the character into the world with a basic collision-resolution step.
- **Text,** by reusing his language's font module with his own renderer.
- **Sound:** a test tone, then loading a WAV file, then a small mixer. Playing two sounds at once is just adding their samples together. He credits Casey Muratori's Handmade Hero series as the starting point for audio.

He writes it in **Jai**, Jonathan Blow's language, which was still listed as an invite-only closed beta in 2026, with no public download, so you can't follow along in it. The same steps work in C or C++ with a library such as SDL. It's a good look at the other end of the spectrum from GameMaker: more control, and a lot more work. randy posts updates at randy.gg, and the video description links his list of learning resources.

1.6 A suggested order

1. **Learn the basics of one language.** Lua (CS50), Python (Pygame), GDScript (Godot) or GML (GameMaker) are all gentle starts.
2. **Remake small classics.** Pong, Breakout and Flappy Bird teach the core loop without design decisions getting in the way. This is exactly how CS50 teaches it.
3. **Pick an engine and follow one full course** to the end (Sections 1.1 to 1.4).
4. **Make your own tiny game,** using the checklist in Section 7.
5. **Join a game jam** (Section 6) and **publish** on itch.io (Section 9).
6. **Then grow:** bigger projects, a second engine, 3D, or building from scratch.

---

2. Choosing an Engine

An **engine** gives you a ready-made editor, rendering, physics, audio, input and exporting, so you can focus on the game itself. A **library** such as Pygame gives you the building blocks but leaves the structure to you. Building **from scratch** means writing most of that yourself.

Option	Cost and license (checked November 2026)	Language	Best at	A good pick if...
Godot (4.7)	Free and open source under the MIT license. No fees or royalties, and your game is yours.	GDScript (Python-like), C#, and C++ for advanced users	Strong, dedicated 2D tools; capable 3D that keeps improving	You want a free, lightweight engine for 2D or mid-sized 3D games
GameMaker	**Free** for non-commercial use. **Professional**, a one-time $99.99, to sell games on desktop, web and mobile. **Enterprise** (a subscription) adds console exports.	GML, as code or as GML Visual blocks	2D games	You want to make 2D games quickly, with a gentle start
Unity	**Personal** is free while your revenue and funding stay under $200,000. **Pro** is $2,310 a year per seat ($210 a month), after a 5% rise in January 2026. The planned Runtime Fee was canceled in September 2024 and never charged.	C#	2D and 3D, mobile games, and a huge library of tutorials and assets	You want one engine for many kinds of games and lots of learning material
Unreal Engine	Free to use. Games owe a **5% royalty** on lifetime gross revenue **above $1 million per product**. Epic's "Launch Everywhere with Epic" program can lower this to 3.5% if the game also releases on the Epic Games Store, and sales on that store are royalty-free.	C++ and Blueprints (visual scripting)	High-end 3D, realistic graphics, large worlds	You want to make 3D games and have a capable PC
Pygame / pygame-ce	Free and open source (LGPL 2.1)	Python	Simple 2D games, and learning how a game loop works	You already know some Python
From scratch	Free. Uses libraries such as SDL for windows, input and sound.	Usually C or C++, but any language works	Learning how engines work; total control	You enjoy low-level programming and are in it for the long haul

Notes on the table:

- **Godot and C#:** C# needs the separate .NET edition of Godot, and Godot 4 can't export C# projects to the web yet. If you want browser games on itch.io, use GDScript.
- **Unreal's royalty:** Epic's license and release pages describe these terms, but Epic's site blocked automated checks, so they were confirmed through search summaries of those pages. Read the current EULA at unrealengine.com/license before you plan around the 3.5% rate.

- **LÖVE**, the framework CS50 uses, is another free 2D option, with Lua.
- **Don't agonize over this choice.** Finishing a small game in any engine teaches you more than comparing engines. You can switch later, and most skills transfer.

---

3. Art and Assets

3.1 Making your own sprites

"How I Design and Animate a Character" (Vimlark, 12 min, 2020) follows a solo game-jam developer making a player character for a platformer, a cute alien, from blank canvas to a full animation set. It's about the art, not the code. His process:

1. **Let a limit decide.** He restricts himself to a four-color palette, Nymph GB by Kerrie Lake, found on **Lospec** (a free palette library) by filtering for exactly four colors. A limit stops you stalling over choices.
2. **Pick a small canvas.** He works at 64 x 64 pixels, but tells beginners to start at 16 x 16, or 32 x 32 at most.
3. **Block out the silhouette** with rough blobs of color, with reference pictures open beside you, then refine.
4. **Lean on familiar shapes.** A big round green head with big eyes reads as "alien" straight away. Big heads and big round eyes read as cute, and pointed eyes made it look mean.
5. **Shade for one light.** He draws characters in three-quarter view, lit from high on one side.

For animation, he keeps the frame count low, as a solo developer: four frames each for **idle** and **run**, one for **jump**, two for **fall**, a **hit** frame plus a copy filled with the lightest color so it flashes, and a **death** frame with X eyes. Two tricks do most of the work:

- **Offset the parts.** In the idle bob, the body drops a pixel, the head follows a frame later, and the antennae a frame after that. Parts that don't move together look connected, not glued.
- **Do squash and stretch in the engine,** by scaling the sprite in code (he uses Construct), instead of drawing extra frames.

He designs in Photoshop out of 20 years' habit, then animates in Aseprite, and says most people should simply use one pixel-art editor for both. The finished character is a free download for personal and commercial games. Credit isn't required, and reselling or redistributing the asset itself isn't allowed.

Tools for pixel art:

- **Aseprite:** a popular paid sprite and animation editor.
- **LibreSprite:** a free, open-source fork of an older version of Aseprite.
- **GIMP:** a free, general-purpose image editor. It works for pixel art with the pencil tool and grid, and for textures and UI.

Tips: pick a small, limited palette and stick to it, keep every sprite at the same scale, and get one character's idle and run looking right before you draw a whole cast.

3.2 Free asset sources, and their licenses

Free assets let you make a complete-looking game while you learn. Always check the license on each asset, not just the site.

Source	What's there	License
Kenney	Thousands of 2D and 3D game assets, UI, fonts and sounds	**CC0** (public domain): free for any use, including commercial. Attribution isn't required, but Kenney's logo is reserved for its own projects.
OpenGameArt	Community-uploaded art, music and sound	**Varies by asset.** Uploads use CC0, CC BY, CC BY-SA, OGA-BY, or GPL/LGPL licenses. Read the license on each asset's page.
Poly Haven	3D models, textures and HDRI lighting	**CC0**: free for any purpose, with credit optional
itch.io game assets	Free and paid asset packs from independent artists	**Set by each creator.** Read each pack's page.

What the common licenses mean (in short):

- **CC0:** do what you like. No credit needed, though it's kind to give it.
- **CC BY and OGA-BY:** free to use, including in commercial games, **if you credit the creator** as they ask.
- **CC BY-SA:** you must credit the creator, and share your changes to that asset under the same license.
- **GPL and LGPL:** written for software. Using GPL art can bring "share-alike" obligations that may reach your game, so be careful in a closed-source or commercial game.
- **"NC" (non-commercial):** you can't use it in a game you sell.

> **Keep a credits file from day one.** For every asset, write down its name, creator, link and license. It makes your credits screen easy and protects you if anyone asks.

3.3 Blender and GIMP

- **Blender** is free, open-source 3D software for modeling, sculpting, texturing, rigging, animation and rendering. Most engines import the `.glb` / `.gltf` and `.fbx` files it exports. For depth, see Learn 3D Modeling, and its section on exporting for games for scale, polygon budgets and LODs. The same models can also be 3D printed: see 3D Printing: Getting Started.
- **GIMP** is a free image editor for textures, sprites and UI art.
- To resize, convert or process many files at once with these tools, see Batch Image and 3D Tools: GIMP, Blender and Pillow.

---

4. How Games Handle Performance

A game has to draw every frame in a few milliseconds: about 16.7 ms at 60 frames per second. A big world has far more detail than any computer could draw that fast, so games cheat, carefully, in ways you don't notice.

4.1 Thousands of trees: level of detail and impostors

"The small detail behind Breath of the Wild's Thousands of Trees - Quick Answer" (Jasper, 1 min, December 2021) answers how *The Legend of Zelda: Breath of the Wild* can show thousands of different trees at once. Jasper makes noclip.website, a browser viewer for game levels, and studies how games render. His answer comes in three layers:

1. **Level of detail** for big objects, such as the towers: from far away, the game draws versions with fewer polygons.
2. **Impostors for distant trees.** Even the lowest-detail tree models are too expensive in those numbers, so very distant trees are swapped for small **pre-rendered pictures** of trees. A whole screen of forest then costs only a handful of triangles, and because they're only used far away, the pictures can be small.
3. **Normal maps are the real trick.** Each impostor comes with a **normal map**, a texture that records which way every part of the surface faces. With it, the engine can relight the flat pictures, so the distant forest matches the time of day, the sun's position and the weather instead of looking pasted on.

Nintendo hasn't published how the game renders its trees, so this rests on Jasper's analysis. The technique itself is standard: Unreal Engine's **Impostor Baker** captures a base color and a normal map for each impostor, and its documentation says most impostors need only those two. Here's the general toolkit it belongs to:

- **Level of detail (LOD):** the game keeps several versions of a model, from the full-detail **LOD0** to simpler **LOD1**, **LOD2** and so on. As an object gets smaller on screen, the game swaps in a simpler version with fewer polygons and materials. A tree that covers a few pixels doesn't need thousands of leaves. Unity's documentation describes LOD as reducing the rendering work as an object gets smaller on screen.
- **Billboards:** at a distance, a 3D object can be replaced by a flat picture of itself on a single rectangle that turns to face the camera. One rectangle is far cheaper to draw than a full mesh.
- **Impostors:** a smarter billboard. The object is rendered ahead of time into textures, often from many angles, and the game shows the view that best matches where the camera is. Storing a normal map (and sometimes depth) alongside the color keeps the lighting convincing, as in the trees above.
- **Hiding the swap:** a sudden switch between levels is called **popping**. Games hide it by switching at a distance where the change is hard to see, or by blending or fading between versions.

4.2 Optimizing for limited hardware

"I Built Cyberpunk 2077 on a Real PS2" (2009aero, 11 min, October 2026) rebuilds a few scenes of *Cyberpunk 2077* as a "demake" for the PlayStation 2: a boss fight, a cutscene, a slice of the city, combat and driving. The PS2 came out in 2000 with a CPU of about 300 MHz, 32 MB of main memory and 4 MB of video memory, so each scene is a lesson in getting away with less:

Problem	What he did
Drawing a city of over 50,000 triangles	Grouped nearby surfaces into chunks, each with a bounding sphere for one quick visibility test. Chunks outside the view are skipped, chunks fully inside take a fast path, and only chunks crossing the edge get careful checks. This is **frustum culling**.
Lighting the streets	**Baked** the fixed street lights before the game runs, as colors stored at each vertex. Only the car's headlights are worked out live, by brightening the road vertices inside their beams. Splitting the road into more triangles gave the light more points, so it looks smoother.
A fully animated cutscene	Worked out the animation in Blender and saved every vertex's position for each sampled pose, packed into 16-bit integers (half the size of 32-bit floats). The PS2 just blends between neighboring poses instead of running the full character rig.
Motion-blur afterimages	Kept the boss's last eight positions, and redrew his current pose where he was three and six frames ago, as fainter transparent copies. The pose is worked out once and reused.
Breakable enemies	Tagged in advance which triangles make up each detachable part. On a hit, the game stops drawing those triangles on the body and throws a separate copy, with a fixed pool of 16 pieces that gets reused.
Sharing an animation between characters	Matched skeleton bones by name and smoothed the motion, then fixed the gaps left by different body proportions in Blender.

He also points to tricks real PS2 games used, such as *God of War* loading the next area while you walk through the current one. And he tested on a real console at the end, because an emulator is great for quick fixes but can behave differently from the actual hardware. It's the same idea as Section 4.1 taken to the extreme: when memory and power are tight, every model, light and animation is simplified, precomputed or cut. It was the channel's first PS2 build, after a series of demakes on the original PlayStation, such as *Skyrim* and *GTA 5*.

4.3 What this means for your first game

- **Don't optimize early.** Small 2D games rarely need it. Make it work, then make it fast if it isn't.
- **Measure before you change anything.** Godot, Unity, Unreal and GameMaker all have built-in **profilers** that show what's taking the time.
- **The usual fixes,** once you need them: draw fewer things (cull what's off screen), draw simpler things far away (LOD and impostors), precompute what never changes (baked lighting), reuse resources instead of creating new ones every frame (like the pool of 16 pieces above), and keep texture sizes sensible. To make LOD versions of your own models in Blender, see Learn 3D Modeling's section on polygon budgets and LOD.

---

5. Fan Remakes, Mods and IP Etiquette

5.1 Skyblivion

"Oblivion Remake Road to Release (Skyblivion Roadmap 2023)" (Rebelzize, 11 min, June 2023) was the 2023 roadmap for **Skyblivion**, a fan-made remake of *The Elder Scrolls IV: Oblivion* (2006) built inside *Skyrim Special Edition*. It's made by volunteers of the TES Renewal project, led by Rebelzize.

The video set a goal of releasing in **2025** and showed, department by department, how far the volunteers had got, partly to recruit more help. It's a rare look at how a huge project tracks its work:

- **The world:** 3,632 of 4,391 landscape cells finished (about 83%), region by region, with 11 of 40 other worldspaces done and most of Oblivion's hellish realms saved for last. After landscaping comes **navmeshing**, the invisible map that lets characters walk around, which was about a third done.
- **Interiors and quests:** 260 of 600 interiors built, focusing on the slow dungeons first, and 97 of 199 quests playable.
- **3D assets:** about 615 objects still to make, with plants and clothing furthest behind. The video says every one of Oblivion's 3D objects has to be **rebuilt from scratch** to meet the legal standard for release, rather than copied from the original game.
- **Music:** all the original tracks kept, plus 17 new ones. The *Shivering Isles* expansion would come after the main release.

Where it stands (checked November 2026):

- **Not released yet.** In December 2025 the team moved its release window from 2025 to 2026. The FAQ still aims for 2026 with no specific date, and the latest showcase on the site, of the Mages Guild questline, was posted in March 2026.
- **Free, but you must own both games.** Its FAQ says you need *Skyrim Special Edition* and *Oblivion GOTY Deluxe* from Steam or GOG. It's for PC only, with no console versions, and the team doesn't take donations.
- **A dispute in October 2026:** the project lead reported that a former volunteer was threatening legal action over code they had contributed. The team says it still plans to release.

5.2 Not the same as Bethesda's official remaster

In April 2025, Bethesda released **The Elder Scrolls IV: Oblivion Remastered**, a paid, official remaster co-developed with Virtuos for Xbox Series X|S, PlayStation 5 and PC. It uses Unreal Engine 5 for its visuals while the original engine still runs the game's logic.

	Skyblivion	Oblivion Remastered
Made by	Volunteer fans (TES Renewal)	Bethesda Game Studios and Virtuos
Status	In development, aiming for 2026	Released April 2025
Price	Free mod, but you must own Skyrim Special Edition and Oblivion	A paid game

	Skyblivion	Oblivion Remastered
Built on	Skyrim Special Edition's Creation Engine	Unreal Engine 5 on top of the original game's engine
Platforms	PC only	Consoles and PC

After the official remaster came out, Bethesda publicly supported community projects, and the Skyblivion team said it would carry on.

5.3 Mod and IP etiquette

Fan projects exist at the rights holder's discretion. Some companies welcome mods and fan work; others don't. In 2016, for example, Nintendo used DMCA takedown notices against AM2R, a fan remake of *Metroid II*, shortly after its release. Projects like Skyblivion avoid many problems by staying free, requiring players to own the original games, and rebuilding assets instead of copying them.

If you make or join a fan project:

1. **Read the publisher's mod or fan-content policy,** and the game's license agreement, before you start.
2. **Never sell it.** Don't charge for a fan game or mod built on someone else's characters, worlds or assets, and be careful with donations and paid early access.
3. **Don't redistribute the original game's files.** Require players to own the game, as Skyblivion does.
4. **Don't imply it's official.** Make it clear who made it, and don't use the company's logos as if you were them.
5. **Credit everyone,** including other modders whose work you build on, and follow their permissions.
6. **Agree on ownership up front.** In a team project, agree in writing how contributions are licensed and what happens if someone leaves.
7. **If you get a takedown notice, comply** and get proper advice. Arguing in public rarely helps.

> **Want to sell something?** Make your own original world and characters. That's the work you can build a business on.

This is general information, not legal advice.

---

6. Game Jams

A **game jam** is a time-limited event where you make a game around a theme, alone or in a team. Jams are the best way to practice finishing, because the deadline forces you to cut scope.

- **itch.io jams:** a directory of hundreds of jams, upcoming and running, for every skill level and engine. The site says anyone can create and host a jam.
- **GMTK Game Jam:** hosted by Game Maker's Toolkit (Mark Brown's YouTube channel), and one of the biggest online jams. It lasts four days, it's open to anyone of any age (under-13s need a parent or

guardian to upload), you can use any engine, and entries must be free to play. The 2026 jam (theme "Count Down") had about 37,000 participants and over 10,500 games. The 2027 jam is listed as starting **August 11, 2027**, with sign-up on itch.io.

- **Global Game Jam:** a yearly jam, usually in late January, held at in-person sites around the world and online. Check its site for the 2027 dates, as reports differ.

Jam tips:

1. Read the rules: theme, time limit, team size, and whether pre-made assets are allowed.
2. Make the smallest version of your idea first, and get it playable in the first quarter of the time.
3. Use free assets (Section 3.2) if the rules allow, and keep your credits file.
4. Leave the last few hours for building, uploading and testing your submission. Uploads fail at the last minute more often than you'd think.
5. Play and rate other entries. That's where you learn the most, and how others find your game.

---

7. Your First Game: a Scope Checklist

The most common reason first games never ship is scope: too big, too many features, too much art. Use this checklist before you start.

- [ ] I can describe the game in one sentence, with **one core mechanic** ("jump between platforms to reach the exit").
- [ ] Someone can **finish it in 5 to 15 minutes.**
- [ ] It has **one level or screen,** or a handful at most.
- [ ] It uses **simple shapes or free assets,** not art I still have to learn to make.
- [ ] It has **no online multiplayer, no open world, and no procedural generation.**
- [ ] It has a **start screen, a way to win or lose, and a way to restart.**
- [ ] I've set a **deadline** (two to four weeks is plenty for a first game).
- [ ] I'll let **at least three people play it** and watch them without helping.
- [ ] I'll **publish it,** even if it isn't perfect (Section 9).

Good first projects: a Pong, Breakout or Flappy Bird remake; a one-screen arcade game like the Pygame course in Section 1.3; a short platformer with three levels; a small puzzle game.

> **Keep an "ideas for later" list.** Every time you think of a great new feature, write it down instead of adding it. Some of it becomes your second game.

---

8. Shipping Your Game

1. **Export from your engine.** Godot, Unity, Unreal and GameMaker can all build for Windows, and most can build for macOS, Linux and the web. A web build lets people play on itch.io without downloading

anything.

2. **For a Python game,** package it with PyInstaller. Packaging Windows Apps with PyInstaller and Inno Setup covers making a `.exe`, bundling your images and sounds, building an installer, and why antivirus tools sometimes flag new builds.
3. **Test on a clean machine,** or at least a different PC, before you share it. Missing files and libraries only show up on a computer that isn't yours.
4. **Include a credits file** with every asset and its license (Section 3.2).
5. **Expect a SmartScreen warning** for unsigned Windows games. The packaging guide explains code signing and how to ship responsibly without it.
6. **Zip it and upload it** to itch.io, or attach it to a GitHub release. Publishing to GitHub with the GitHub CLI shows how to create releases.
7. **Write a short page:** what the game is, controls, a few screenshots or a GIF, and the credits.

---

9. Publishing on itch.io and Steam

9.1 itch.io

itch.io is the easiest place to publish a first game, a jam entry or a free download.

- **It costs nothing to use.** There's no upload fee.
- **You choose itch.io's share.** It uses "open revenue sharing": the default is 10%, and you can set anything from 0% to 100% in your seller settings.
- **Payment processing** through PayPal or Stripe typically costs $0.30 plus 2.9% per sale, so itch.io suggests a price of at least $2. Its own example: a $10 sale at the default 10% leaves the seller $8.41.
- **Tax:** with itch.io's own payouts, itch.io is the merchant of record and collects VAT for you. If you're paid directly, VAT is your responsibility.

9.2 Steam

Selling on Steam goes through **Steamworks**, Valve's developer site.

- **The Steam Direct fee** is **$100 per game**, paid before release. It isn't refundable, but you get it back once that game earns $1,000 in adjusted gross revenue on Steam.
- **Onboarding:** you sign Valve's agreements online and provide bank, tax and identity information.
- **Waiting periods for your first few games:** at least **21 days** between paying the fee and releasing, and a public **"Coming Soon" page for at least two weeks** before launch, so people can wishlist it.
- **Review:** Valve checks your store page and your build before each goes live, which takes 1 to 5 days.
- **Valve's share:** the standard revenue share is widely reported as 30%, with lower rates on a game's revenue above $10 million and $50 million. Check your Steam Distribution Agreement for the exact terms.

> **Plan your Steam page early.** Wishlists build up while your "Coming Soon" page is live, so most developers put it up months before launch, not the minimum two weeks.

---

10. Starting a Game Company: the Basics

You don't need a company to put a free game on itch.io. When you start earning money, a few basics matter wherever you live:

1. **Keep records from your first sale:** income, fees, and what you spend on software, assets and hardware.
2. **Learn your tax obligations.** Steam and itch.io both ask for tax information before paying you. Selling to other countries can involve VAT or sales tax.
3. **Consider a business structure** when the money becomes real. Rules differ by country, so check your government's official small-business guidance or ask an accountant.
4. **Put agreements in writing** with anyone who works on your game: who owns the code, art and music, and how money is shared. This matters even between friends.
5. **Check your game's name** isn't already taken by another game or trademark before you build a brand on it.
6. **Keep every license** for engines, fonts, music and assets, and make sure each one allows commercial use.
7. **Budget for marketing, not just development.** A store page, a trailer, screenshots and time talking to players are part of the work.

This is general information, not legal or tax advice. A local accountant or lawyer can tell you what applies to you.

---

11. Keep Learning: Courses, Docs, Talks and Postmortems

11.1 Official engine learning

- **Godot:** the documentation includes a full beginner tutorial, "Your first 2D game" ("Dodge the Creeps"), with finished code in GDScript and C#.
- **Unity:** Unity Learn offers free courses and pathways, including **Junior Programmer**.
- **Unreal Engine:** Epic's learning library has free courses and tutorials.
- **GameMaker:** free tutorials and the manual.
- **pygame-ce:** the documentation starts with a basic game loop example.

11.2 Free books and articles

- **Game Programming Patterns** by Robert Nystrom: a book on structuring game code (game loops, components, state and more), free to read online.

- **Red Blob Games** by Amit Patel: free, interactive explanations of pathfinding, hexagonal grids, procedural map generation and other game math.
- For the math itself (vectors, trigonometry, matrices), see Section 7 of Computer Science Fundamentals.

11.3 GDC talks and postmortems

The **Game Developers Conference (GDC)** publishes many of its talks for free.

- **GDC Vault, free content:** free talks from every year back to the 1990s. Some talks need a paid membership.
- **GDC Festival of Gaming on YouTube:** GDC's YouTube channel, with hundreds of talks.
- **Postmortems** are talks or articles where developers explain what went right and wrong. GDC's "Classic Game Postmortem" series is a good place to start, for example **"'Doom:' A Classic Game Postmortem"** (GDC Festival of Gaming, 59 min).

11.4 Channels to follow

- **Game Maker's Toolkit:** videos on game design, and the home of the GMTK Game Jam.
- **Brackeys:** beginner-friendly tutorials, now focused on Godot.
- **freeCodeCamp.org:** long, free courses, including new Godot 4 courses.

---

12. Common Mistakes

Mistake	Why it hurts	Do this instead
Starting with your dream game	It's too big to finish, so you lose motivation	Make three tiny games first, then grow (Section 7)
Switching engines every few weeks	You keep relearning basics and never finish	Commit to one engine for your first few games
Watching tutorials without building	It feels like progress, but you can't do it alone	After each tutorial, change it or make something similar from scratch
Using assets without checking the license	You may have to remove them, or face a takedown	Prefer CC0 assets and keep a credits file (Section 3.2)
Optimizing before anything works	Time spent on speed you don't need yet	Make it work, measure with a profiler, then fix what's slow (Section 4.3)
Never showing anyone	You miss problems that every new player finds	Let people play early and watch them
Polishing forever	The game never ships	Set a deadline and publish (Sections 8 and 9)

---

13. Sources

Courses and tutorials

- CS50's Introduction to 2D Game Development (Harvard): https://cs50.harvard.edu/games/
- CS50 2D Games FAQs (certificates and deadline): https://cs50.harvard.edu/games/faqs/
- CS50's Introduction to Game Development, archived 2018 version: https://cs50.harvard.edu/games/2018/
- CS50x (Harvard): https://cs50.harvard.edu/x/
- freeCodeCamp, "Create a Arcade-Style Shooting Game with Python and PyGame": https://www.freecodecamp.org/news/create-a-arcade-style-shooting/
- freeCodeCamp, "Create a 3D RPG Game with Godot": https://www.freecodecamp.org/news/create-a-3d-rpg-game-with-godot/
- Clear Code, "The ultimate introduction to Pygame": https://www.youtube.com/watch?v=AY9MnQ4x3zk
- Godot, "Your first 2D game": https://docs.godotengine.org/en/stable/getting_started/first_2d_game/index.html
- Unity Learn: https://learn.unity.com/
- Epic Developer Community, Unreal Engine learning: https://dev.epicgames.com/community/unreal-engine/learning
- GameMaker tutorials: https://gamemaker.io/en/tutorials
- GameMaker manual: https://manual.gamemaker.io/
- GameMaker manual, GML Visual overview: https://manual.gamemaker.io/lts/en/Drag_And_Drop/Drag_And_Drop_Overview/DnD_Overview.htm
- pygame-ce documentation: https://pyga.me/docs/
- pygame-ce on GitHub (license and history): https://github.com/pygame-community/pygame-ce
- LÖVE: https://love2d.org/
- Game Programming Patterns: https://gameprogrammingpatterns.com/
- Red Blob Games: https://www.redblobgames.com/
- Handmade Hero (Casey Muratori): https://handmadehero.org/
- randy's learning resources (linked from his video): https://github.com/bigrando420/resources/wiki
- Jai (programming language), Wikipedia (closed beta status): https://en.wikipedia.org/wiki/Jai_(programming_language)
- GDC Vault, free content: https://gdcvault.com/free

Videos from the original notes

- freeCodeCamp.org, "Pokémon Coding Tutorial - CS50's Intro to Game Development": https://youtu.be/gx_qorHxBpI
- Vimlark, "How I Design and Animate a Character": https://youtu.be/FnkDXMssMPU
- Jasper, "The small detail behind Breath of the Wild's Thousands of Trees - Quick Answer": https://youtube.com/shorts/sh6s17WnWBM
- randy, "Programming an entire game from scratch": https://youtu.be/O-TAjnvWjAE

- Rebelzize, "Oblivion Remake Road to Release (Skyblivion Roadmap 2023)": https://youtu.be/H1q1nXLDjB8
- Pirate Software, "Start Making Games": https://youtube.com/shorts/vhs83wzZ32Y
- 2009aero, "I Built Cyberpunk 2077 on a Real PS2": https://youtu.be/RaL9GRliHWg

Other videos

- GDC Festival of Gaming, "'Doom:' A Classic Game Postmortem": https://www.youtube.com/watch?v=NnkCujnYNSo

Engines, licenses and pricing

- Godot license: https://godotengine.org/license/
- Godot documentation (4.7): https://docs.godotengine.org/en/stable/
- Godot, C# basics and platform support: https://docs.godotengine.org/en/stable/tutorials/scripting/c_sharp/c_sharp_basics.html
- GameMaker plans: https://gamemaker.io/en/get
- Unity pricing updates (Runtime Fee cancellation, Personal threshold, Pro price): https://unity.com/products/pricing-updates
- Unreal Engine licensing options: https://www.unrealengine.com/license
- Unreal Engine release page (Launch Everywhere with Epic): https://www.unrealengine.com/release

Performance

- Unity Manual, Level of detail (LOD): https://docs.unity.com/en-us/engine/current/manual/analysis/graphics-performance-profiling/lod/level-of-detail
- Unity Manual, Billboard Renderer: https://docs.unity.com/en-us/engine/current/manual/analysis/graphics-performance-profiling/lod/2d-images-lod/class-billboard-renderer
- Unreal Engine, Impostor Baker plugin (base color and normal maps): https://dev.epicgames.com/documentation/en-us/unreal-engine/impostor-baker-plugin-in-unreal-engine
- noclip.website (Jasper's game-level viewer): https://noclip.website/
- PlayStation 2 technical specifications (Wikipedia): https://en.wikipedia.org/wiki/PlayStation_2_technical_specifications

Art and assets

- Kenney support and license: https://kenney.nl/support
- OpenGameArt: https://opengameart.org/
- Poly Haven license: https://polyhaven.com/license
- Lospec palette list: https://lospec.com/palette-list
- Lospec, Nymph GB palette by Kerrie Lake: https://lospec.com/palette-list/nymph-gb
- Vimlark, "Cute Animated Alien" (free asset and its terms): https://vimlark.itch.io/cute-animated-alien

- LibreSprite on GitHub: https://github.com/LibreSprite/LibreSprite
- Blender: https://www.blender.org/
- GIMP: https://www.gimp.org/
- Creative Commons licenses: https://creativecommons.org/share-your-work/cclicenses/

Fan remakes

- Skyblivion: https://skyblivion.com/
- Skyblivion FAQ: https://skyblivion.com/faq/
- Skyblivion (Wikipedia): https://en.wikipedia.org/wiki/Skyblivion
- PC Gamer, "Elder Scrolls Skyblivion mod faces lawsuit from former volunteer": https://www.pcgamer.com/games/the-elder-scrolls/elder-scrolls-skyblivion-mod-faces-lawsuit-from-former-volunteer-project-lead-urges-please-dont-turn-this-into-any-kind-of-witch-hunt/
- Bethesda, "The Elder Scrolls IV: Oblivion Remastered Available Now!": https://bethesda.net/en/article/4HJix0Dpayy0OVgZiTmlRW/the-elder-scrolls-iv-oblivion-remastered-launch
- Bethesda, Oblivion Remastered: https://elderscrolls.bethesda.net/en-US/oblivion-remastered
- PC Gamer, "Nintendo issues DMCA notice against that really good Metroid II fan remake": https://www.pcgamer.com/nintendo-issues-dmca-notice-against-that-really-good-metroid-ii-fan-remake

Game jams

- itch.io jams: https://itch.io/jams
- GMTK Game Jam: https://gamemakerstoolkit.com/jam
- GMTK Game Jam 2027 on itch.io: https://itch.io/jam/gmtk-2027
- Global Game Jam: https://globalgamejam.org/

Publishing

- itch.io creator FAQ: https://itch.io/docs/creators/faq
- itch.io, getting paid and VAT: https://itch.io/docs/creators/payments
- Steamworks, getting started: https://partner.steamgames.com/doc/gettingstarted
- Steamworks, Steam Direct fee: https://partner.steamgames.com/doc/gettingstarted/appfee
- Steamworks, onboarding and waiting periods: https://partner.steamgames.com/doc/gettingstarted/onboarding

---

Latest version: github.com/ImmuneMoon/Learn-Game-Development

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.